TECHNICAL SPECIFICATION

Batch 1-out-of-2 Deniable Commitment Scheme

Proof of Concept. Dynamic Tree Topologies for $V \in [2 \dots 20]$ and Depths $\in [10 \dots 19]$

Author: Anton A. Sokolov (acmxddk@gmail.com)

October 2026

This work is licensed under CC BY 4.0

Post-Quantum Security Level: 128-bit Soundness (NIST Level 1 Compliance)
Cryptographic Primitive: SHA-256 (32 bytes / 256 bits)
Core Purpose: This protocol extends the 1-out-of-2 Deniable Plain Scalar Bit-Commitment Scheme into a high-throughput, batch-processed framework. A Prover publishes a single cryptographic commitment and subsequently presents V pairs of 256-bit scalars to a Verifier. The verification routine establishes with 128-bit post-quantum security that, for each of the V presented pairs, the Prover committed to at least one of the two constituent scalars, while the alternative scalar remains arbitrary. The protocol achieves zero-knowledge deniability, i.e., the Verifier gains zero information regarding which specific scalar in each pair was committed to. To reduce bandwidth overhead, the protocol aggregates all V instances into a single batch, optimizing the internal Merkle tree via GGM hierarchical salt derivation and a C-floor prefix leaf cascade.

---

1 Public Parameters and Architecture

The execution environment and structural scaling of the batching commitment pipeline are governed by configuration variables and deterministic structural constraints.

1.1 Scheme Public Parameters

The cryptographic suite operates using the following publicly known configuration variables:

- V: The total number of parallel scalar pairs processed within the batch context.
- D: The target balanced binary cryptographic tree depth.
- X: The public mixing seed providing high-entropy, verifiable randomness used to shuffle bit tracks.
- C: The count of floors embedded into the lower leaf-cascade layer. While the cascading floor optimization logic is elaborated in subsequent algorithmic sections, C acts as the uniform grouping parameter determining how many raw bits form a single base leaf chunk. The public parameter C defaults to 5 across this draft.

1.2 Dependent Structural Equations

From the selected values of the public configuration tuple (V, D, X, C), the following variables are strictly computed:

- N: The total bitwise footprint of the global super-batch array, calculated as:

$$N = C \times 2^D \tag{1}$$

- W: The mandatory individual bitstream length required for each instance to maintain structural alignment, derived via:

$$W = \frac{N}{V} \tag{2}$$

1.3 Commitment Stage & Pseudo-Random Bit Mixing

During the initialization of the commitment sequence, data arrays are mapped through an 'oblivious' permutation layer:

1. The Prover accepts V secret 256-bit scalars ($\{S_i\}$ for $i \in [0 \dots V-1]$) as private inputs.
2. The Prover expands each private scalar S_i via a deterministic cryptographic hash function into an individual, flat W-bit bitstream (T_i).
3. The Prover treats all individual T_i tracks as a single N-bit batch bitstream (B). The assignment of every individual bit from each track T_i to its destination coordinate inside B is handled by a deterministic mixing algorithm.
4. This mixing algorithm relies strictly and exclusively on the public randomness of the seed X, ensuring that bit destination indexing is oblivious to and independent of the actual underlying data bit values. The mixing algorithm ensures that the bits from each track T_i are distributed deterministically and uniformly across the global super-array B. Although, the requirement for uniformity can be relaxed.

1.4 Leaf Chunk Processing & Merkle Tree Commitment

The Prover steps linearly through the resulting un-padded global super-array B. The bitstream is sliced into sequential, non-overlapping C-bit chunks (where $C = 5$ bits per chunk), denoted as $\{C_i\}$ for $i \in [0 \dots 2^D - 1]$. For each individual chunk encountered, the Prover executes the following commitment routine:

1. For each individual chunk C_i encountered, the Prover generates a single cryptographic leaf node L_i by hashing the chunk's 5 bits with a 256-bit random mask M_i.
2. Through this iterative process, the Prover obtains exactly 2^D leaves.
3. The accompanying random masks $\{M_i\}$ are generated deterministically via the Prover's internal GGM tree stemming from the Prover's private, securely guarded GGM root key K.
4. The Prover builds a balanced binary Merkle tree of depth D over the computed leaves $\{L_i\}$.
5. Finally, the Prover publishes the resulting 32-byte Merkle tree root to the Verifier as the public, immutable commitment to the private array of scalars $\{S_i\}$.
6. **Cascade Architecture Bridge:** Each leaf node L_i is computed as the final, top-tier output of a nested 5-floor sequential prefix hashing cascade to securely bind the underlying data bits directly to the mask.

2 Proof Generation Protocol

The response sequence executes zero-knowledge deniability through symmetric stream collisions, utilizing the deterministic bit-value-agnostic mixing pipeline to construct a challenge space.

2.1 Scalar Pairing and Dual Vector Separation

1. The Prover picks V scalar pairs $\{P_i\}$ such that, for each index $i \in [0 \dots V-1]$, exactly one scalar in the pair P_i is the originally committed secret scalar S_i from the secret vector $\{S_i\}$. The alternative scalar in the pair is arbitrary.

2. The Prover puts all V first scalars from the $\{P_i\}$ pairs into a vector denoted as $\{S_i^0\}$. Concurrently, all V second scalars from the $\{P_i\}$ pairs are compiled into a complementary vector denoted as $\{S_i^1\}$.

2.2 Super-Array Construction and Alignment Properties

1. The Prover executes the exact deterministic bitstream expansion function over every element in both vectors, producing two distinct sets of expanded W-bit tracks.

2. The Prover builds two global batch super-arrays, denoted as B_0 and B_1, by processing the tracks generated from $\{S_i^0\}$ and $\{S_i^1\}$ respectively, following the identical mixing layout parameters specified in Section 1.3.

3. **Track Symmetry Rationale:** Because the baseline mixing algorithm is entirely bit-value-agnostic and fully deterministic based solely on X, each i-th scalar pair shares an identical structural track footprint across both super-arrays. The bits from the first scalar map into B_0 at the exact same geographical offsets where the bits from the second scalar map into B_1. This structural alignment means a scalar pair shares a common, public track layout regardless of which slot contains the true secret variable.

2.3 The XOR Coincidence Filter and Candidate Pool Generation

1. The Prover evaluates a bitwise XOR operation combining the two completed public super-arrays to produce a dedicated intersection map, denoted as the XOR-super-array (B_X):

$$B_X = B_0 \oplus B_1 \tag{3}$$

2. **Deniability Rationale:** The specific bit positions where B_X evaluates to exactly zero indicate identity coordinates where both tracks match. Consequently, these bit values can be revealed to the Verifier without leaking info about the true positions of the secret scalars within their respective pairs.

3. The Prover slices the resulting array B_X into non-overlapping 5-bit chunks using the exact same structural boundary logic utilized for array B.

4. For each resulting chunk $C_{X,i}$ computed from array B_X, the Prover evaluates a prefix weight denoted as the rank (CR_i), computed strictly as the total count of leading zeros present in the chunk.

5. **Candidate Pool Admissions:** The Prover collects candidate coordinates into a unified set denoted as the Candidate Pool (CP). In accordance with multi-phase optimization rules, complete chunks ($CR_i = 5$) represent zero-penalty configurations. Incomplete chunks ($CR_i \in [1 \dots 4]$) are admitted into CP if and only if they contain a valid prefix bit matching a track T_i that is currently below the target soundness threshold. Chunks with $CR_i = 0$ are permanently excluded.

2.4 Multi-Phase Adaptive Selector Logic

The Prover passes the Candidate Pool CP into an advanced multi-phase deterministic optimization routine that isolates a specific subset of candidate indices such that the structural byte-cost is minimized while ensuring each independent track T_i contains no fewer than 128 opened bits. This routine balances the structural savings of opening incomplete chunks against the cryptographic byte-cost of their respective floor stub penalties.

3 Verification Protocol

The Verifier executes an identical, deterministic challenge reconstruction loop to isolate the open-bit indices blindly, evaluating the proof payload against the public scalar pairs without processing any private state.

3.1 Challenge Space and Candidate Pool Reconstruction

1. The Verifier enters the loop already possessing the public 32-byte Merkle Root commitment. It receives the V scalar pairs $\{P_i\}$ from the Prover alongside the transmitted Proof Payload.

2. The Verifier runs the exact same Candidate Pool CP builder and chunk selection algorithms as the Prover.

3. Specifically, the Verifier executes the dual vector separation steps outlined in Section 2.1, the super-array construction and mixing steps specified in Section 2.2, the XOR filter and ranking steps from Section 2.3, and the multi-phase deterministic optimization selector routine.

4. **Oblivious Consistency Principle:** Because all tracking, filtering, and selection constraints depend exclusively on public parameters (B_X, CP, and the public seed X), both the Prover and Verifier possess an identical, non-distinguishing view of the available search space. Consequently, the Verifier executes the exact same multi-constraint selection loops locally and blindly reconstructs the identical set of opened chunk indices. Transmission of index maps over the wire is entirely bypassed.

3.2 Cryptographic Reconstruction and Root Validation

1. Having isolated the exact opened bit positions, the Verifier reads the corresponding data bits directly from the reconstructed super-array B_0 (or interchangeably from B_1, as the bit values at these specific verified coordinates are guaranteed to be identical).

2. The Verifier hashes the opened 5-bit chunks with the GGM leaf masks derived from the un-pruned node keys provided in the Proof Payload, running them through the 5-floor cascade logic to reconstruct the opened Merkle tree leaves $\{L_i\}$.

3. Finally, the Verifier computes upward hashing rounds using the reconstructed leaves alongside the complimentary Merkle tree path hashes supplied in the Proof Payload. The Verifier asserts that the final computed root matches the public commitment root exactly. If they intersect perfectly, the batch proof is authenticated as valid.

4 Cascade C-Floor Leaf Structure ("Floors of Leaves")

The base level of the commitment tree employs a C-stage (say, 5-stage) sequential hashing cascade nested by the prefix principle. Each leaf index k maps uniquely to a specific 5-bit chunk

$(B_{5k} \dots B_{5k+4})$ extracted from the global super-array B, bound sequentially to a private 256-bit GGM mask M_k.

When an incomplete chunk ($CR_i < 5$) is selected, only its verified prefix bits (matching its leading zero rank) are read from the data stream, and the remaining unaligned block space is securely bypassed by publishing the precise intermediate floor hash stub $(L_{\text{floor}_1} \dots L_{\text{floor}_4})$ as a local bandwidth penalty. The global Merkle commitment tree is built exclusively over Floor 5 leaves (L_{floor_5}).

4.1 Optimization Synergy Case Study (Dual-Engine Compression)

To illustrate the simultaneous compound bandwidth savings generated by combining GGM tree pruning with prefix cascade stub bypassing, let us examine an isolated case study tracing three adjacent opened chunks:

1. **Aclat-Mask Bypass (Chunks with $CR_k < 5$):** Suppose chunk k_1 evaluates to a partial rank $CR_{k_1} = 3$. This means only its first three prefix bits match across B_0 and B_1, while bits 4 and 5 collide. The Verifier does not require a GGM leaf mask M_{k_1} because the private execution path at Floor 1 is completely unreached. The Prover securely hides the non-coincident bits by shipping the pre-computed intermediate hash stub L_{floor_2,k_1} inside the flat vector array `floorHashStubs`. The Verifier reads the three clean prefix bits from the stream, couples them with the payload stub, and directly computes Floor 3 upward without needing any private key material.

2. **GGM Subtree Compression (Chunks with $CR_k = 5$):** Suppose the next two adjacent blocks, chunk k_2 and chunk k_3, are complete rank-5 matches. Both require their full, independent 256-bit GGM leaf masks (M_{k_2} and M_{k_3}) to recompute Floor 1. Because these chunks are binary siblings sitting under a shared GGM parent node at level $D-1$, the Prover completely omits both 32-byte leaf keys from the payload. Instead, it publishes a single internal GGM node key root fragment from level $D-1$. The Verifier reads this parent token from `revealedGgmKeys` and derives both leaf masks locally, achieving a 50% tree path bandwidth compression ratio across that section.

5 Candidate Pool Construction and Selection Algorithms

The bandwidth profile optimization is achieved via a formalized, two-phase algorithmic pipeline executed symmetrically by both the Prover and the Verifier. This pipeline consists of the Candidate Pool (CP) Builder and a multi-phase Selection Engine.

5.1 The Candidate Pool Builder Algorithm

The Candidate Pool (CP) serves as the bounded search space of index coordinates from which verification blocks are harvested. Rather than searching the entire tree space (2^D slots), the engine restricts candidates based on the structural alignment of the public scalar tracks.

Step 1: Track-Wise Extraction. The Verifier (and Prover) uses the public mixing seed X to isolate and reconstruct the exact coordinates of every bit belonging to each independent track T_i ($i \in [0 \dots V-1]$) within the global super-arrays B_0 and B_1.

Step 2: Chunk Identity Filtering. The engine scans the XOR-super-array $B_X = B_0 \oplus B_1$ sequentially in non-overlapping C-bit steps ($C = 5$). For each chunk index $k \in [0 \dots 2^D - 1]$:

- The rank CR_k (the count of leading zeros) is computed.
- If $CR_k = 5$ (Zero-Penalty Configuration), the block index k is immediately pushed into the baseline Candidate Pool CP.

- Advanced Extension: If $CR_k \in [1 \dots 4]$ (Partial Prefix Configuration), the block index k is admitted into CP if and only if it contains a valid prefix bit matching a track T_i that is currently below the 128-bit soundness quota. Blocks with $CR_k = 0$ are permanently excluded.

5.2 The Two-Phase Structural Compaction Selection Engine

The selection engine treats proof assembly as a multi-stage combinatorial search over a directed acyclic graph (the binary Merkle and GGM tree paths) to fulfill the 128-bit track quotas while minimizing structural branch splits.

Step 1: Target Bit Quota Tracking. The engine initializes an entropy counter vector Q of length V, where $Q[i] = 0$ for all $i \in [0 \dots V-1]$. The algorithm terminates successfully when:

$$\forall i \in [0 \dots V-1], \quad Q[i] \geq 128 \tag{4}$$

Step 2: Phase-Split Execution. The allocation sequence runs over a dual-phase timeline to reconcile local compaction with global Shannon efficiency path consolidation:

1. **Phase 1: Hierarchical Macro Sub-Tree Pre-Mining (Exploration Stage):** While the accumulated aggregate entropy is below the mid-point transition threshold ($\sum Q[i] < \frac{128 \times V}{2}$), the engine groups candidates inside hierarchical sub-tree neighborhoods. It scores each unselected chunk k via an empirical macro-density block function:

$$\text{Score}(k) = [\Lambda_4(k) \times (1.0 + \gamma(k) \times 0.65)] + [\Lambda_{32}(k) \times 0.15] + [\rho(k) \times 0.05] \tag{5}$$

   Where $\Lambda_4(k)$ is the rank sum of the immediate 4-leaf quad-cousin block, $\gamma(k)$ is the count of already selected leaves inside that quadrant, $\Lambda_{32}(k)$ is the aggregate density of the wider 32-leaf window, and $\rho(k) = CR_k$.

2. **Phase 1a: Symmetrical Sibling Sweep (Forced Twin Arbitrage):** Immediately following Phase 1 iterations, before standard costs are evaluated, any unselected chunk whose immediate binary twin sibling is already selected is instantly committed if its rank > 0. Because the path footprint for that branch is already paid for by the first twin, opening the sibling reclaims 1 to 4 bits of entropy at zero additional cryptographic payload cost.

3. **Phase 2: Shannon Entropy Path Consolidation (Consolidation Stage):** Upon passing the mid-point threshold, the engine switches to a Global Marginal Path Cost Strategy. In each iteration, it computes a dynamic cost-efficiency score (E_k) for every unselected block $k \in CP$:

$$E_k = \frac{\Delta\text{Entropy}_k}{\Delta\text{Bandwidth}_k} \tag{6}$$

   Where $\Delta\text{Bandwidth}_k = (\Delta\text{GGM Keys}_k \times 32) + (\Delta\text{Merkle Hashes}_k \times 32) + \text{Floor Penalty}_k$. To eliminate high-level branch fracturing, $\Delta\text{Merkle Hashes}_k$ implements a **Layer-Weighted Modifier**. If a choice splits a pristine, completely dark branch, its cost value scales linearly with depth ($+(D - \text{currentLevel}) \times 1.5$), forcing the engine to consume residual gaps within already fractured branches instead of cracking open clean sub-trees.

5.3 Coordinate-Invariant Tie-Breaking

If multiple candidate blocks achieve the exact same maximum efficiency score, the engine enforces a strict numerical tie-breaker:

$$\text{Selected Index} = \min(k_1, k_2, \dots) \tag{7}$$

The block with the lowest absolute geographical index inside the super-array B is selected. This guarantees that the Prover and Verifier converge on an identical choice layout, completely independent of input processing sequence bias or vector sorting permutations.

6 Technical Criteria for Post-Quantum Security

1. **Zero-Knowledge Privacy & Deniability:** Because the verifier only evaluates indices where the bitstreams match identically, the verification mathematical loop is completely symmetric. The Verifier is securely convinced that at least one scalar per pair is bound to the public root, but receives exactly zero information pointing to which one was committed. Complete deniability is preserved.

2. **Confidentiality (Data Privacy):** The remaining unopened bit states remain strictly confidential. Extracting a hidden bit from an un-coincident or un-challenged block requires a quantum brute-force search via Grover's algorithm against the 256-bit state of an unrevealed cascade floor hash stub, forcing a maximum complexity barrier of $\sqrt{2^{256}} = 2^{128}$ operations.

3. **Unforgeability (Binding):** The rigorous collision resistance of the SHA-256 Merkle tree guarantees that a malicious Prover cannot present fraudulent scalar pairs and manipulate the coincidence pool to pass verification without altering the root (2^{128} post-quantum collision resistance).

4. **Verification 'Paradox Invariance':** Higher-level companion hashes for completely dark branches must be provided explicitly rather than inferred from public data alone. Because unopened leaf nodes are anchored by secret GGM keys at their base, computing them locally from scratch without providing stubs would leak GGM key material, violating the deniability and zero-knowledge privacy invariants of the scheme.

5. **Uncollapsible GGM Boundaries:** Complete GGM node compression requires that both child sub-branches consist entirely of maximum-rank matches ($\rho = C$). Isolated maximum-rank chunks whose binary twins possess a lower rank cannot be collapsed up-tree and must expose their leaf keys to preserve protocol verification correctness.

7 Theoretical Optimality Bounds & System Convergence

The protocol reaches structural stabilization at deep execution profiles, achieving a verified transmission spending limit of 20.85 bytes/bit at $V = 16,\ D = 15,\ C = 9$.

The convergence of the selection engine is mathematically bounded by the inherent entropy distribution of the pseudo-random track mixing layer. Let $\mathcal{R}_C$ represent the total count of maximum-rank chunks ($\rho = C$) generated across the global super-array. The total number of revealed keys within the GGM subtree is strictly constrained by the relation:

$$\text{Total GGM Keys} = 2 \times \mathcal{R}_{\text{collapsed}} + \mathcal{R}_{\text{stranded}} \tag{8}$$

Where $\mathcal{R}_{\text{collapsed}}$ denotes the pairs of complete matches residing under a common ancestor node at level $D - 1$, and $\mathcal{R}_{\text{stranded}}$ represents isolated complete matches whose binary siblings possess a partial prefix alignment ($\rho < C$).

Because the verification loop requires the GGM mask to expand the prefix cascade for complete matches while hiding non-coincident paths via intermediate floor stubs, the stranded leaf-layer keys constitute a theoretical floor bound. The Two-Phase Selection Engine achieves a globally optimal compaction footprint matching this boundary constraint, ensuring absolute protocol correctness and cryptographic consensus under any track sorting permutation.